# Estudo Dirigido 1

1º Bimestre

## Descrição

O objetivo deste Estudo Dirigido é a prática dos conceitos iniciais de Programação Orientada a Objetos, com foco na **criação de classes**, **definição de atributos**, uso de **construtores** e implementação de **métodos simples**.

## Instruções de entrega

Para a submissão do trabalho, siga os passos abaixo:

- Criar um novo repositório no github chamado **programacao-oo**
  - Selecionar para criar o REAME.md
  - Selecionar para criar .gitgnore usando template Dart
- Incluir o **nome dos integrantes** da equipe no README.md (até 3 alunos)
- Compartilhar o do repositório na **planilha** disponibilizada pelo professor (TEAMS)

Para garantir organização, o trabalho deve seguir a estrutura de diretórios abaixo:

```
📁 estudo-dirigido-1.1
│
├── 📄 pubspec.yaml
│
└── 📁 lib
    ├── 📁 questao01
    │   ├── 📄 main.dart
    │   └── 📄 pessoa.dart
    │
    ├── 📁 questao02
    │   ├── 📄 main.dart
    │   └── 📄 produto.dart
    │
    ├── 📁 questao03
    │   ├── 📄 main.dart
    │   └── 📄 carro.dart
    │
    └── ...
```

# Critérios de avaliação

A avaliação será baseada nos seguintes aspectos:

- Uso correto de classes e atributos
- Implementação de construtores
- Implementação de métodos
- Organização e legibilidade do código
- Capacidade de explicar o funcionamento do código

# Questões

## Questão 1 – Pessoa

Crie uma classe Pessoa com os seguintes atributos obrigatórios:

- String nome
- int idade

Implemente um construtor que receba todos esses valores. Depois, crie um método exibirDados() que mostre as informações da pessoa de forma organizada. No main(), instancie exatamente 3 pessoas diferentes e exiba os dados de cada uma.

**Restrições:**

- nome não pode ser vazio
- idade deve ser maior ou igual a 0

## Questão 2 - Relógio

Crie uma classe Relogio com os seguintes atributos obrigatórios:

- int hora
- int minuto
- int segundo

Implemente um construtor que receba todos esses valores. Depois, crie um método exibirHorario() que mostre o horário no formato HH:MM:SS. No main(), instancie exatamente 3 relógios diferentes e exiba o horário de cada um.

**Restrições:**

- hora deve estar entre 0 e 23
- minuto deve estar entre 0 e 59
- segundo deve estar entre 0 e 59

## Questão 3 - Data

Crie uma classe Data com os seguintes atributos obrigatórios:

- int dia
- int mes
- int ano

Implemente um construtor que receba todos esses valores. Depois, crie um método exibirData() que mostre a data no formato DD/MM/AAAA. No main(), instancie exatamente 3 datas diferentes e exiba cada uma.

**Restrições:**

- dia deve estar entre 1 e 31
- mes deve estar entre 1 e 12
- ano deve ser maior que 0

## Questão 4 - Lâmpada

Crie uma classe Lampada com os seguintes atributos obrigatórios:

- bool ligada
- int quantidadeLigacoes
- bool queimada

Implemente um construtor que inicialize a lâmpada como desligada, com zero ligações e não queimada. Crie os seguintes métodos:

- ligar() → liga a lâmpada
- desligar() → desliga a lâmpada
- exibirEstado() → mostra se está ligada, desligada ou queimada

No main(), instancie 1 lâmpada e simule seu uso, ligando e desligando várias vezes.

**Restrições:**

- após 5 vezes ligando, a lâmpada deve ficar queimada
- uma lâmpada queimada não pode mais ser ligada
- a cada vez que ligar, incremente quantidadeLigacoes

## Questão 5 - Aluno

Crie uma classe Aluno com os seguintes atributos obrigatórios:

- String nome
- double nota1
- double nota2

Implemente um construtor que receba todos esses valores. Crie os seguintes métodos:

- calcularMedia() → retorna a média das notas
- verificarAprovacao() → informa se o aluno foi aprovado ou reprovado
- exibirBoletim() → mostra nome, notas, média e situação

No main(), instancie exatamente 10 alunos diferentes e exiba o boletim de cada um.

**Restrições:**

- notas devem estar entre 0 e 10
- aluno é aprovado se média ≥ 6
- média deve ser calculada corretamente

## Questão 6 - Produto

Crie uma classe Produto com os seguintes atributos obrigatórios:

- int codigo
- String nome
- double precoUnitario
- int quantidade

Implemente um construtor que receba todos esses valores. Crie os seguintes métodos:

- calcularDesconto() → retorna o valor do desconto aplicado
- calcularTotal() → retorna o valor total da compra com desconto
- exibirResumo() → mostra código, nome, preço unitário, quantidade, desconto e total

No main(), instancie 1 produto e exiba o seu resumo de cada. Em seguida, substitua os atributos deste produto e o exiba novamente.

Regras de desconto por quantidade:

- até 4 unidades: sem desconto
- de 5 a 9 unidades: 5% de desconto
- de 10 a 19 unidades: 10% de desconto
- 20 unidades ou mais: 15% de desconto

**Restrições:**

- codigo deve ser maior que 0
- nome não pode ser vazio
- precoUnitario deve ser maior que 0
- quantidade deve ser maior que 0
- o total deve considerar a quantidade e o desconto aplicado

## Questão 7 - Contador

Crie uma classe Contador com um construtor que inicialize o contador com um valor inicial. Crie os seguintes métodos:

- incrementar() → aumenta o valor em 1
- decrementar() → diminui o valor em 1
- zerar() → redefine o valor para 0
- exibirValor() → mostra o valor atual do contador

No main(), instancie exatamente 2 contadores diferentes e realize algumas operações com cada um.

**Restrições:**

- o valor inicial deve ser maior ou igual a 0
- o valor não pode ficar negativo
- ao decrementar, se o valor estiver em 0, ele deve permanecer em 0

## Questão 8 - Endereço

Crie uma classe Endereco com os seguintes atributos obrigatórios:

- String rua
- int numero
- String bairro
- String cidade
- String estado
- int cep

Implemente um construtor que receba todos esses valores. Depois, crie um método exibirEndereco() que mostre todas as informações do endereço de forma organizada. No main(), instancie exatamente 3 endereços diferentes e exiba cada um.

**Restrições:**

- rua não pode ser vazia
- numero deve ser maior que 0
- bairro não pode ser vazio
- cidade não pode ser vazia
- estado não pode ser vazio
- cep não pode ser vazio
- cep deverá ser impresso no formado XXXXX-XXX

## Questão 9 - Pessoa

Crie uma classe Pessoa com os seguintes atributos obrigatórios:

- String nome
- int idade
- String cpf
- Endereco endereco

Implemente um construtor que receba todos esses valores. Depois, crie um método exibirPessoa() que mostre os dados da pessoa de forma organizada, incluindo os dados do endereço. No main(), instancie exatamente 3 pessoas diferentes, cada uma com 1 endereço, e exiba os dados completos de cada uma.

**Restrições:**

- nome não pode ser vazio
- idade deve ser maior ou igual a 0
- cpf não pode ser vazio
- endereco deve ser do tipo Endereco

## Questão 10 - Quadrado

Crie uma classe Quadrado com os seguintes atributos obrigatórios:

- double lado
- String caractere

Implemente um construtor que receba todos esses valores. Crie os seguintes métodos:

- calcularArea() → retorna a área do quadrado
- calcularPerimetro() → retorna o perímetro do quadrado
- desenhar() → desenha o quadrado usando o caractere informado
- exibirResumo() → mostra lado, área e perímetro
- ehIgual(outroQuadrado) → retorna true se os dois quadrados forem iguais

No main(), instancie exatamente 2 quadrados diferentes e 2 quadrados iguais, exiba o resumo de cada um e desenhe ambos. Verifique quais quadrados são iguais.

**Restrições:**

- lado deve ser maior que 0
- caractere não pode ser vazio
- o caractere usado no desenho deve ter apenas 1 símbolo
- o desenho deve formar um quadrado proporcional ao valor de lado

## Questão 11 - Triângulo

Crie uma classe Triangulo com os seguintes atributos obrigatórios:

- double lado1
- double lado2
- double lado3
- String caractere

Implemente um construtor que receba todos esses valores. Crie os seguintes métodos:

- calcularPerimetro() → retorna o perímetro do triângulo
- calcularArea() → retorna a área do triângulo usando a fórmula de Heron
- desenharEsquerda() → desenha um triângulo alinhado à esquerda
- desenharCentralizado() → desenha um triângulo centralizado
- exibirResumo() → mostra os lados, a área e o perímetro

Também crie métodos acessores para alterar as dimensões do triângulo. No main(), instancie exatamente 2 triângulos diferentes, exiba o resumo de cada um e desenhe ambos nas duas formas pedidas.

**Restrições:**

- lado1, lado2 e lado3 devem ser maiores que 0
- os três lados devem formar um triângulo válido
- caractere não pode ser vazio
- o caractere usado no desenho deve ter apenas 1 símbolo

## Questão 12 - Círculo

Crie uma classe Circulo com os seguintes atributos obrigatórios:

- double raio
- String cor

Implemente um construtor que receba todos esses valores.

Crie os seguintes métodos:

- calcularArea() → retorna a área do círculo
- calcularPerimetro() → retorna o perímetro do círculo
- exibirResumo() → mostra raio, cor, área e perímetro

No main(), instancie exatamente 3 círculos diferentes e exiba o resumo de cada um.

**Restrições:**

- raio deve ser maior que 0
- cor não pode ser vazia
- use 3.14 como valor de π e declare como constante

## Questão 13 - Pedido

Crie uma classe Pedido com os seguintes atributos obrigatórios:

- int numero
- Produto produto
- int quantidade

Implemente um construtor que receba todos esses valores. Crie os seguintes métodos:

- calcularTotal() → calcula o valor total do pedido
- exibirPedido() → mostra o número do pedido, os dados do produto, a quantidade e o total

No main(), instancie exatamente 2 pedidos diferentes, cada um com um produto, e exiba os dados completos.

**Restrições:**

- numero deve ser maior que 0
- produto deve ser do tipo Produto
- quantidade deve ser maior que 0
- o total deve ser calculado usando o preço do produto

## Questão 14 - Carro

Crie uma classe Motor com os seguintes atributos obrigatórios:

- int potencia
- String tipo

Crie uma classe Carro com os seguintes atributos obrigatórios:

- String modelo
- Motor motor

Implemente os construtores para ambas as classes. Crie os seguintes métodos:

- exibirMotor() → mostra os dados do motor
- exibirCarro() → mostra o modelo do carro e os dados do motor

No main(), instancie exatamente 2 carros diferentes, cada um com um motor, e exiba os dados completos.

**Restrições**:

- modelo não pode ser vazio
- motor deve ser do tipo Motor
- potencia deve ser maior que 0
- tipo não pode ser vazio

## Questão 15 - Cofrinho

Crie uma classe Cofrinho com os seguintes atributos obrigatórios:

- double saldo
- int quantidadeDepositos

Implemente um construtor que inicialize os valores. Crie os seguintes métodos:

- depositar(double valor) → adiciona valor ao saldo
- quebrar() → mostra o total acumulado e zera o cofrinho
- exibirSaldo() → mostra o saldo atual

No main(), instancie exatamente 1 cofrinho, faça vários depósitos e depois quebre o cofrinho.

**Restrições:**

- saldo inicial deve ser maior ou igual a 0
- quantidadeDepositos deve iniciar com 0
- o valor do depósito deve ser maior que 0
- ao quebrar, o saldo deve voltar para 0
- a quantidade de depósitos deve ser incrementada corretamente a cada depósito

## Questão 16 - Personagem

Crie uma classe Personagem com os seguintes atributos obrigatórios:

- String nome
- int vida
- int ataque

Implemente um construtor que receba todos esses valores. Crie os seguintes métodos:

- atacar(Personagem inimigo) → reduz a vida do inimigo
- exibirStatus() → mostra nome, vida e ataque
- estaVivo() → retorna se o personagem ainda está vivo

No main(), crie exatamente 2 personagens e simule um combate entre eles.

**Restrições:**

- nome não pode ser vazio
- vida deve ser maior que 0
- ataque deve ser maior que 0
- a vida não pode ficar negativa
- um personagem com vida 0 não pode atacar

## Questão 17 - Semáforo

Crie uma classe Semaforo com os seguintes atributos obrigatórios:

- String corAtual - represente com emojies 🔴🟡🟢
- int tempoRestante

Implemente um construtor que inicialize o semáforo com uma cor inicial e um tempo. Crie os seguintes métodos:

- trocarCor() → altera a cor seguindo a ordem: vermelho → verde → amarelo → vermelho
- reduzirTempo() → diminui o tempo restante em 1 unidade
- exibirEstado() → mostra a cor atual e o tempo restante

No main(), instancie exatamente 1 semáforo e simule seu funcionamento por alguns ciclos.

Regras de tempo:

- vermelho: 5 unidades
- verde: 4 unidades
- amarelo: 2 unidades

**Restrições:**

- corAtual deve assumir apenas: "vermelho", "verde" ou "amarelo"
- quando o tempo chegar a 0, o semáforo deve trocar automaticamente para a próxima cor
- ao trocar de cor, o tempo deve ser reiniciado conforme a regra da nova cor

## Questão 18 - Playlist + Música

Crie uma classe Musica com os seguintes atributos obrigatórios:

- String titulo
- String artista
- int duracaoSegundos

Crie uma classe Playlist com os seguintes atributos obrigatórios:

- String nome
- Musica musica1
- Musica musica2
- Musica musica3

Implemente os construtores para ambas as classes. Crie os seguintes métodos:

- exibirPlaylist() → mostra o nome da playlist e os dados das músicas
- calcularDuracaoTotal() → retorna a duração total da playlist
- tocarPlaylist() → exibe as músicas sendo tocadas em sequência

No main(), instancie exatamente 1 playlist com 3 músicas diferentes e exiba todas as informações.

**Restrições:**

- nome da playlist não pode ser vazio
- todas as músicas devem ser do tipo Musica
- duracaoSegundos deve ser maior que 0
- deve somar corretamente o tempo total

## Questão 19 - Casa + Cômodo

Crie uma classe Comodo com os seguintes atributos obrigatórios:

- String nome
- double tamanho

Crie uma classe Casa com os seguintes atributos obrigatórios:

- String proprietario
- Comodo comodo1
- Comodo comodo2
- Comodo comodo3

Implemente os construtores para ambas as classes. Crie os seguintes métodos:

- exibirCasa() → mostra o proprietário e os dados dos cômodos
- calcularAreaTotal() → retorna a soma das áreas dos cômodos

No main(), instancie exatamente 1 casa com 3 cômodos diferentes e exiba todas as informações.

**Restrições:**

- proprietario não pode ser vazio
- todos os cômodos devem ser do tipo Comodo
- tamanho deve ser maior que 0
- a área total deve ser calculada corretamente

## Questão 12 - Time Pokémon

Crie uma classe Pokemon com os seguintes atributos obrigatórios:

- int numero
- String nome
- String tipo
- int nivel
- int hp

Crie uma classe Treinador com os seguintes atributos obrigatórios:

- String nome
- Pokemon pokemon1
- Pokemon pokemon2
- Pokemon pokemon3

Implemente os construtores para ambas as classes. Crie os seguintes métodos:

- exibirTime() → mostra o nome do treinador e os dados dos pokémons
- calcularPoderTotal() → soma o nível de todos os pokémons
- pokemonMaisForte() → exibe o pokémon com maior nível

No main(), instancie exatamente 1 treinador com 3 pokémons diferentes e exiba todas as informações.

**Restrições:**

- nome do treinador não pode ser vazio
- todos os pokémons devem ser do tipo Pokemon
- nivel deve estar entre 1 e 100
- hp deve ser maior que 0
- tipo não pode ser vazio